Sclera Document Chatbot

AI Features Documentation

Intelligent question-answering system for HVAC equipment — combining PDF documentation, live sensor data, and LLM reasoning

April 2026 | Port 8004 | FastAPI

GPT-4-Turbo / GPT-4o-mini / text-embedding-3-large / LlamaIndex / AWS S3

Where AI Is Used

Area	AI Role
PDF Indexing	Converts document pages into searchable vector embeddings
Query Understanding	Determines the best source for each question
Context Validation	Checks if live sensor/work order data is sufficient to answer
Hybrid Detection	Decides when to combine documentation + live data
Response Generation	Streams natural language answers to users

Table of Contents

1 Project Overview

2 AI Features

2.1 PDF Document Ingestion & Vector Indexing

2.2 Intelligent Query Routing & Response Generation

2.3 Semantic Similarity Scoring

2.4 Context Sufficiency Validation

2.5 Hybrid Query Detection

3 System Architecture

4 Implementation Details

5 Improvements & Future Enhancements

1 · Project Overview

What Is This Project?

The Sclera Document Chatbot is an intelligent question-answering system built specifically for HVAC (Heating, Ventilation, and Air Conditioning) equipment. It allows users to ask natural language questions about specific HVAC units and receive accurate, contextually relevant answers.

What Problem Does It Solve?

HVAC technicians and engineers often need to look up technical information from large, complex PDF manuals or interpret live sensor and operational data. This system eliminates the need to manually search through hundreds of pages of documentation by:

- Automatically finding the right answer from indexed technical PDFs
- Interpreting real-time asset data (sensor readings, work orders)
- Combining both sources intelligently when needed
- Falling back to GPT's general knowledge when local data is unavailable

2 · AI Features

Feature 1: PDF Document Ingestion & Vector Indexing

Purpose

Converts raw HVAC PDF manuals uploaded from S3 into a searchable vector index stored on disk. Once indexed, any query about that equipment can search these documents semantically (by meaning, not just keywords).

Endpoint

```
POST /api/fetch-files
```

Workflow

1. Validate request — all URLs must end with .pdf
2. Sanitize manufacturer + model names → create asset_id: '{manufacturer}_{model}'
3. For each S3 URL (async): download PDF bytes, save to temp file
4. Extract pages using LlamaIndex SimpleDirectoryReader (preserves page_number metadata per chunk)
5. Check if vector index already exists for this asset → load existing or create new
6. Parse document into chunks using SimpleNodeParser (chunk size: 1000 tokens, overlap: 200 tokens)
7. Embed each chunk using text-embedding-3-large (OpenAI)
8. Insert embedded chunks into vector index and persist to disk at ./app/storage/{asset_id}/

Technologies Used

LlamaIndex — RAG framework for indexing and retrieval | OpenAI text-embedding-3-large — high-accuracy embedding model | SimpleNodeParser — semantic document chunking | AWS S3 (boto3) — PDF source storage | asyncio.to_thread — non-blocking background processing

Enhancements

- Index reuse — checks for existing index before rebuilding; avoids re-processing already-indexed documents
- 200-token chunk overlap preserves context at chunk boundaries
- Async processing — file ingestion runs in the background so the API responds instantly
- Page numbers tracked per chunk for source attribution

Failure Handling

S3 download failure → HTTP 400. PDF parsing failure → HTTP 500. Individual file failure → warning logged, other files continue. Non-PDF URLs → rejected with HTTP 400 before any processing.

Feature 2: Intelligent Query Routing & Response Generation

Purpose

The core AI brain of the system. When a user asks a question, this feature decides the best source to answer from — live asset data, indexed PDF documents, or a combination of both — and streams the response back in real time.

Endpoint

```
POST /api/query
```

Routing Workflow

1. Step 1 — Semantic Similarity Guardrail: embed query + context using text-embedding-3-small; if cosine similarity < 0.1 → mark context irrelevant
2. Step 2 — Context Sufficiency Check: send query + context to GPT-4o-mini with YES/NO prompt; if YES → stream context-only response
3. Step 3 — Load PDF Index: load vector index from ./app/storage/{manufacturer}_{model}/; if missing → continue without PDF

4. Step 4 — PDF Relevance Check: retrieve top 3 chunks; if highest score ≥ 0.3 → PDF is relevant; if < 0.3 AND context irrelevant → return 'not found'
5. Step 5 — Hybrid Query Detection: check for temporal keywords and numeric overlap between query and context
6. Step 6 — Route to one of four paths: Context Only / PDF Only / Hybrid / GPT Fallback

Response Paths

Path	Trigger Condition	Model Used
Context Only	Context has exact values for the query	GPT-4-Turbo
PDF Only	PDF relevant, context irrelevant	GPT-4-Turbo + LlamaIndex
Hybrid	Both sources relevant, hybrid indicators found	GPT-4-Turbo + LlamaIndex
GPT Fallback	No index available	GPT-4-Turbo (general knowledge)
Not Found	No source has relevant information	Static message

Technologies Used

GPT-4-Turbo — primary response generation (temperature: 0, max tokens: 2048) | GPT-4o-mini — YES/NO context sufficiency check (max tokens: 5) | text-embedding-3-small — fast real-time semantic similarity | LlamaIndex VectorStoreIndex — PDF retrieval (top-k: 5) | FastAPI StreamingResponse — real-time token delivery

Failure Handling

PDF query engine failure → caught with try/except; falls back to GPT response. No relevant information found → structured 'not found' message. OpenAI API errors → HTTP 500. Authentication failure → HTTP 401.

Feature 3: Semantic Similarity Scoring

Purpose

A lightweight utility that measures how topically related a user's query is to the provided asset context. Used as a guardrail to prevent the LLM from being misled by irrelevant data.

Workflow

1. Embed the query text using text-embedding-3-small → 1536-dim vector
2. Embed the context text using text-embedding-3-small → 1536-dim vector
3. Compute cosine similarity: dot(query_vec, context_vec) / (|query_vec| × |context_vec|)
4. Return normalized float score (0.0 = unrelated, 1.0 = identical meaning)

Technologies Used

OpenAI text-embedding-3-small — lightweight, fast embedding model | NumPy — vector math

Feature 4: Context Sufficiency Validation

Purpose

Uses a fast, cheap LLM call to determine whether the provided asset context contains the exact data values needed to answer the user's query — without needing to consult PDF documentation.

Workflow

1. Construct a YES/NO prompt: 'Does this context contain exact data values to answer: {query}? Only answer YES if specific numbers, IDs, or dates are literally present.'
2. Call GPT-4o-mini with temperature=0, seed=42, max_tokens=5
3. Parse response: if 'yes' in response → return True, else return False

Technologies Used

GPT-4o-mini — minimal, cost-efficient model for binary classification. Configured with seed=42 for reproducibility. Max tokens capped at 5.

Feature 5: Hybrid Query Detection

Purpose

Automatically identifies queries that need both live asset data and PDF documentation to be answered correctly — for example, 'Is the temperature reading I'm seeing normal for this unit?'

Workflow

1. Keyword scan — check query for temporal/real-time indicators: ['current', 'right now', 'at this moment', 'today', 'actual reading'] → if found: return True
2. Numeric overlap check — extract all numbers from context and query using regex; if any numbers overlap → user is referencing live data → return True
3. If neither check triggers: return False

Technologies Used

Python re (regex) — number extraction from strings

Edge Cases

- Query mentions a number that coincidentally matches context → treated as hybrid (safe default, richer answer)
- No numbers in either query or context → falls through to keyword check only

3 · System Architecture

Overview

The system follows a three-tier RAG (Retrieval-Augmented Generation) architecture with intelligent routing logic at its core. There are two main pipelines:

- Ingestion Pipeline — indexes PDFs into a persistent vector store
- Query Pipeline — routes user questions to the optimal information source

Ingestion Pipeline

Step	Component	Technology
1. Download	S3 PDF Download	AWS S3 + boto3
2. Parse	PDF Page Extraction	LlamaIndex SimpleDirectoryReader
3. Chunk	Document Chunking	SimpleNodeParser (1000 tokens, 200 overlap)
4. Embed	Vector Embedding	OpenAI text-embedding-3-large
5. Store	Persist to Disk	./app/storage/{asset_id}/

Query Pipeline

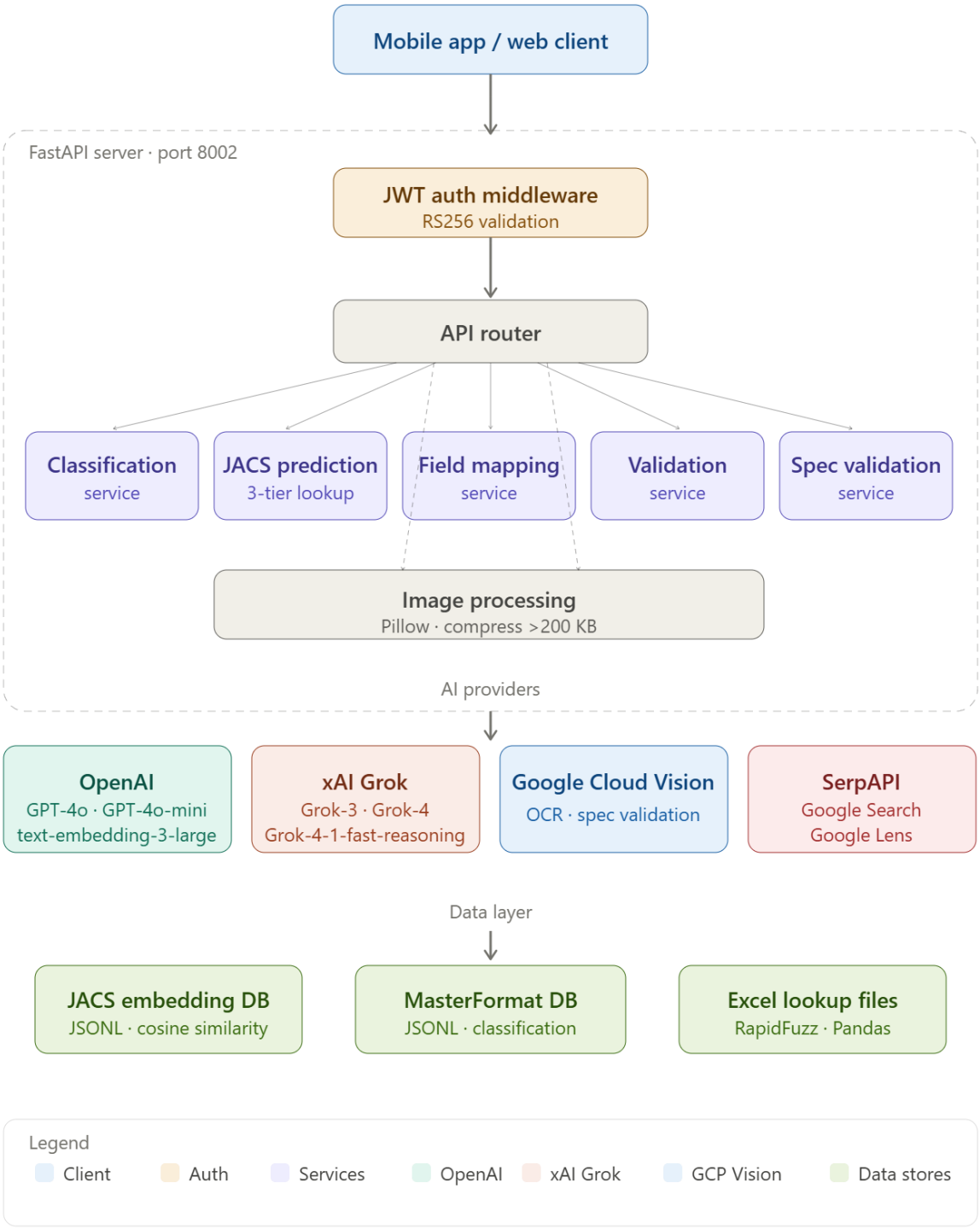
Step	Component	Technology
1. Auth	JWT Middleware	RS256 + JWKS from BFF server
2. Similarity	Semantic Similarity Check	text-embedding-3-small + NumPy (threshold: 0.1)
3. Context Check	Sufficiency Validation	GPT-4o-mini YES/NO (max_tokens=5)
4. PDF Retrieval	Vector Index Search	LlamaIndex top-5 (threshold: 0.3)
5. Detection	Hybrid Query Detection	Regex + keyword scan
6. Generation	Streaming Response	GPT-4-Turbo + FastAPI StreamingResponse

Response Routing Decision Logic

Condition	Result
Context similarity ≥ 0.1 AND GPT-4o-mini says YES	Stream Context-Only Response
PDF relevance score ≥ 0.3, context irrelevant	Stream PDF-Only Response
Both PDF relevant + context relevant, hybrid indicators detected	Stream Hybrid Response
No PDF index exists	GPT-4-Turbo Fallback (general knowledge)
PDF score < 0.3 AND context irrelevant	Return 'Not Found' Message

System Architecture Diagram

The diagram below illustrates the complete system flow — from the client through JWT authentication, the ingestion and query pipelines, all five response paths, and final streaming delivery back to the client.


Mobile app / web client
FastAPI server · port 8002
JWT auth middleware
RS256 validation
API router
Classification
service
JACS prediction
3-tier lookup
Field mapping
service
Validation
service
Spec validation
service
Image processing
Pillow · compress >200 KB
AI providers
OpenAI
GPT-4o · GPT-4o-mini
text-embedding-3-large
xAI Grok
Grok-3 · Grok-4
Grok-4-1-fast-reasoning
Google Cloud Vision
OCR · spec validation
SerpAPI
Google Search
Google Lens
Data layer
JACS embedding DB
JSONL · cosine similarity
MasterFormat DB
JSONL · classification
Excel lookup files
RapidFuzz · Pandas
Legend
Client
Auth
Services
OpenAI
xAI Grok
GCP Vision
Data stores

4 · Implementation Details

Key Design Decisions

Multi-source Fallback Chain

The system checks three sources in priority order: live context → PDF documentation → GPT general knowledge. This ensures the user always gets an answer, even for edge cases where documentation is missing or context is empty.

Two-tier Embedding Model Strategy

text-embedding-3-large is used during document indexing (runs once, offline) for maximum accuracy. text-embedding-3-small is used during real-time query routing (runs on every request) for speed and cost efficiency.

Deterministic LLM Behavior

Response generation uses temperature=0 and seed=42 for the context sufficiency classifier. This prevents non-deterministic YES/NO answers for the same input, which is critical for consistent routing behavior.

Streaming-First Responses

All response paths use FastAPI StreamingResponse. This provides immediate feedback to users even for long answers, improving perceived performance significantly. Controlled 10–50ms delays between chunks ensure smooth delivery.

Persistent Vector Index

The vector index is written to disk once and reused across requests. Re-indexing is only triggered when new PDFs are added. This eliminates redundant embedding API calls and startup latency.

Async Background Processing

PDF ingestion is executed in a background thread via asyncio.to_thread. The API returns 200 OK immediately, preventing client timeouts for large document sets.

Important Logic Patterns

Pattern	Location	Description
Similarity threshold guards	llm.py	0.3 for PDF relevance, 0.1 for context relevance
Chunk overlap	llm.py	200-token overlap to preserve cross-boundary context
OpenAI retry config	llm.py	max_retries=2 on OpenAI client instances
Numeric overlap detection	file_service.py	Regex extracts numbers from both query and context
Global LlamaIndex settings	llm.py	Settings.embed_model set globally to prevent model mismatch on retrieval
Sanitized asset IDs	file_service.py	Manufacturer + model sanitized via [^a-z0-9\-_] → _

System Prompts

Prompt	Used When	Instruction to LLM
PROMPT_CONTEXT_ONLY	Context has exact values	Use only sensor/work order data, include work order numbers
PROMPT_PDF_ONLY	PDF relevant, no useful context	Use retrieved document chunks only, do not mix in context
PROMPT_HYBRID	Both sources relevant	Combine document knowledge with live readings, cite work orders

Prompt	Used When	Instruction to LLM
GENERAL_LLM_SEARCH_PROMPT	PDF query (no prompt override)	Answer from technical documentation with precision
generate_llm_prompt()	No index available	Answer from GPT general HVAC knowledge

5 · Improvements & Future Enhancements

Performance Improvements

Improvement	Description	Impact
Prompt caching	Cache repeated system prompts with OpenAI's prompt caching feature	50%+ latency reduction on repeated queries
Batch embedding	Embed multiple document chunks in a single API call during ingestion	Fewer API calls, faster indexing
Parallel PDF processing	Process multiple PDFs concurrently using asyncio.gather	Faster ingestion for many documents
Index warming	Pre-load frequently accessed indexes into memory at startup	Eliminates cold-start disk reads
Async BFF calls	Replace synchronous requests with httpx for token validation	Non-blocking authentication

Accuracy Improvements

Improvement	Description
Reranking	Add a cross-encoder reranker (e.g., Cohere Rerank) after initial vector retrieval to improve chunk selection
Dynamic thresholds	Make similarity thresholds (0.3, 0.1) configurable per deployment instead of hardcoded
Query expansion	Generate 2–3 query reformulations before retrieval to improve recall for ambiguous questions
Metadata filtering	Filter retrieved chunks by document section type (e.g., troubleshooting, specifications)
Hybrid search	Combine BM25 keyword search with vector search for improved precision on exact model numbers

Scaling Ideas

Idea	Description
Cloud vector store	Replace local SimpleVectorStore with Pinecone or Weaviate for multi-instance deployments
Distributed ingestion	Move PDF processing to a message queue (SQS + Lambda) to handle bursts of ingestion requests
Centralized index storage	Store vector indexes in S3 rather than local disk to share across multiple API instances
Request caching	Cache responses for identical (query, asset_id) pairs using Redis with a short TTL
Health check endpoint	Add /health and /ready endpoints for load balancer and container orchestration support

Additional Feature Ideas

- Multi-asset querying — allow a single query to search across multiple HVAC units simultaneously
- Source citations — return page number and document name alongside the answer for auditability
- Conversation memory — maintain per-session message history so users can ask follow-up questions
- Non-PDF support — extend ingestion pipeline to handle Excel (.xlsx), Word (.docx), and image-based documents via OCR
- Confidence scoring — return a confidence level with each response so the frontend can display uncertainty indicators
- Audit logging — persist all queries, routing decisions, and responses to a database for compliance and debugging